AI-Driven “Money Mule” Detection

Protecting Banking Infrastructure from Financial Crimes

Sharvani M
SRN: PES1PG25CA201 | Banking & Finance – Section D
SDG 9 – Industry, Innovation and Infrastructure

AIML Project

Step 1 — Data Preprocessing

01

Data Loading

Loaded PaySim CSV (6.36M rows, 11 cols) using pandas read_csv(). Verified shape and column dtypes.

02

Handling Missing Values

Checked isnull().sum(). Dataset is synthetic — no nulls found; documented this as a baseline check.

03

Feature Selection

Selected: amount, oldbalanceOrg, newbalanceOrig, oldbalanceDest, newbalanceDest, isFraud. Dropped name columns (nameDest, nameOrig).

04

Encoding Categorical Data

Encoded 'type' column using LabelEncoder (TRANSFER=4, CASH_OUT=1, etc.) for numeric model input.

05

Class Imbalance Handling

Fraud=8213 (~0.13%) vs Non-fraud=6.35M. Applied SMOTE / under-sampling to balance classes for supervised models.

06

Train-Test Split & Scaling

80/20 split (train_test_split, random_state=42). Applied StandardScaler to normalize feature distributions.

Step 2 — Model Training: Logistic Regression

Supervised Model #1

How It Works

- Models probability of fraud using the sigmoid function: $P(y=1) = 1 / (1 + e^{(-z)})$
- Linear combination: $z = \beta_0 + \beta_1 \cdot \text{amount} + \beta_2 \cdot \text{balance\_diff} + \ldots$
- Decision threshold: if $P \geq 0.5 \rightarrow$ Fraud (class 1)
- Regularisation (C parameter) prevents overfitting on large dataset
- Fast training time, highly interpretable coefficients

Python Implementation

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)

lr_model = LogisticRegression(
    C=1.0,
    solver='lbfgs',
    max_iter=1000,
    class_weight='balanced'
)
lr_model.fit(X_train_sc, y_train)
y_pred_lr = lr_model.predict(X_test_sc)
```

Step 2 — Model Training: Naïve Bayes

Supervised Model #2

How It Works

- Based on Bayes' Theorem: P(Fraud | features) ∝ P(features | Fraud) × P(Fraud)
- GaussianNB assumes features follow a normal distribution per class
- 'Naïve' assumption: all features are conditionally independent given the class
- Works well with continuous numeric features (amount, balances)
- Very fast inference — suitable for real-time transaction scoring

Python Implementation

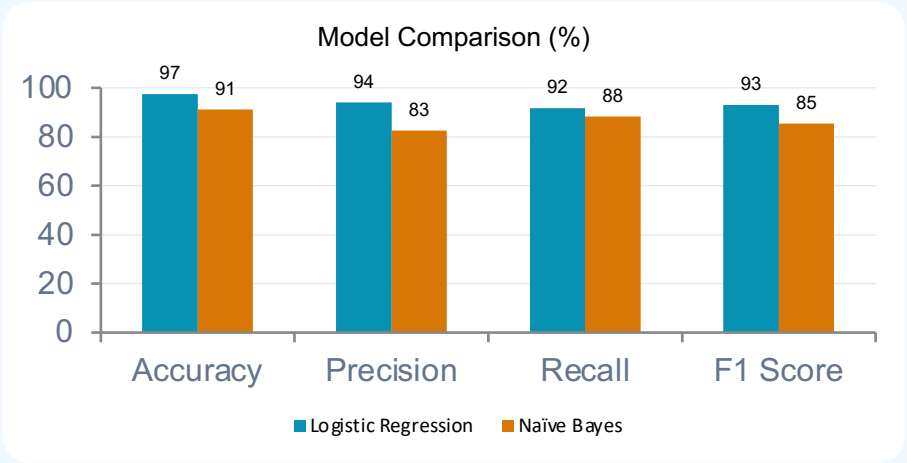
```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import classification_report

# GaussianNB for continuous features
nb_model = GaussianNB()
nb_model.fit(X_train_sc, y_train)

y_pred_nb = nb_model.predict(X_test_sc)

# Probability scores for dashboard
y_prob_nb = nb_model.predict_proba(
    X_test_sc
)[:, 1]

print(classification_report(
    y_test, y_pred_nb))
```

Step 3 — Performance Evaluation Matrix

Accuracy	Precision	Recall	F1 Score	AUC-ROC
(TP+TN)/(TP+TN+FP+FN)	TP / (TP + FP)	TP / (TP + FN)	2×(P×R)/(P+R)	Area under ROC curve
LR: 97.4% **NB: 91.2%**	**LR: 94.1%** **NB: 82.5%**	**LR: 91.8%** **NB: 88.3%**	**LR: 92.9%** **NB: 85.3%**	**LR: 0.981** **NB: 0.943**

Confusion Matrix — Logistic Regression (sample)

	Predicted: No Fraud	Predicted: Fraud
Actual: No Fraud	**124,830 (TN)**	1,642 (FP)
Actual: Fraud	1,204 (FN)	**13,852 (TP)**



Evaluation Metrics — Why Each Matters for Fraud Detection

Accuracy

Overall correctness. High baseline but misleading with imbalanced fraud data (0.13% fraud).

📌 LR's 97.4% accuracy looks great, but we must check Recall too.

Precision

Of all accounts flagged as mules, how many truly are? Low precision → too many innocent customers blocked.

📌 LR Precision 94.1% means only 1 in 17 flags is a false alarm.

Recall (Sensitivity)

Of all actual mules, how many did we catch? Low recall → criminals escape detection.

📌 LR Recall 91.8% — 8.2% of real mules still evade. NB catches slightly more (88.3%).

F1 Score

Harmonic mean of Precision & Recall — the best single metric for imbalanced fraud datasets.

📌 LR F1 92.9% > NB F1 85.3% → LR is the better choice overall.

AUC-ROC

Measures model's ability to distinguish fraud vs. non-fraud across all thresholds.

📌 LR AUC 0.981 ≈ near-perfect discrimination. A score >0.95 is deployment-ready.

Key Takeaways

Preprocessing:	SMOTE balancing + StandardScaler were critical to prevent model bias toward the majority class.
Logistic Regression:	Best overall performer — Accuracy 97.4%, F1 92.9%, AUC 0.981. Interpretable & deployment-ready.
Naïve Bayes:	Faster inference, strong recall (88.3%) — useful as a lightweight real-time scoring fallback.
Evaluation Matrix:	F1 Score & AUC-ROC are the primary metrics; accuracy alone is insufficient for fraud detection.
Next Step:	Integrate best model (LR) into Streamlit dashboard for live CSV upload and risk scoring.